

Hooks:-

Hooks is a special feature that let you use React features like state, lifecycle methods, context, ref, effects etc inside functional components.

Common hooks:-

useState :- store/change data
useEffect :- run side effects
useRef :- store mutable values or access DOM elements
useContext
useMemo
useCallback

1) useEffect :-

useEffect is used to perform side effects

side effects means:-

- fetching API data
- Updating the DOM
- Timers(setInterval)
- Event Listeners
- local storage operations

syntax :-

```
React.useEffect(() => {  
  // code here  
  
  return(){  
    //clean up code (optional)  
  }  
}, [Dependency array])
```

Type of useEffects:-

1) Runs every render:-

```
useEffect(() => {  
  console.log("Runs every render");  
}) // No dependency Array
```

2) Runs only once:-

```
useEffect(() => {
  console.log("Runs once")
},[]); // Empty dependency array
```

3) Runs when dependency changes:-

```
useEffect(() => {
  console.log("Runs every time count changes");
},[count]); // Dependency array with a value
```

=====

useRef:-

useRef is used for Accessing DOM elements and storing values without re-rendering.

Difference between useState and useRef:-

changing the value in useRef doesn't re-render component,
but changing the value in useState forces a re-render component

Example :-

```
function App(){

  const inputRef = React.useRef(null);

  /*
  {
    current : input
  }
  */

  React.useEffect(() => {
    inputRef.current.focus();
  },[]);

  return (
    <>
    <input type = "email" ref = {inputRef} placeholder = "email"/>
    <input type = "password" placeholder = "password"/>
    </>
  )
}
```

